Deep Dive 06 | Earthquake Game and Learning

Study the Phaser drill lifecycle and distinguish it from the separate quiz project.

React/Vite Edition | Repository-grounded study material | Snapshot: 2026-10-24

1. Where the game lives

The interactive earthquake drill is a standalone browser game under frontend/public/earthquake_drill. index.html loads Phaser 3 from a CDN and game.js defines the game configuration and scene behavior. It is distinct from the separate earthquake-quiz folder, which has its own React/Vite app and dependencies.

2. Phaser lifecycle

- A Phaser.Game is created from the CONFIG object.
- PreloadScene loads local assets or creates visual elements needed by later scenes.
- PlayScene creates the player, environment, controls, timer and falling-object behavior.
- Update logic reads keyboard state, moves the player and checks collisions with hazards or cover.
- ResultScene presents the outcome when the timer or collision condition ends the drill.

3. Controls and game mechanics

The drill uses keyboard directions to move and a duck/cover action based on Shift or Space, as shown in the project instructions. The player should reach protective cover during the earthquake sequence and avoid falling objects. Collision checks use Phaser geometry helpers. The game loop and scene state run entirely in the browser; no server persistence is implied unless a separate API call is implemented.

4. Learning objective

The simulation provides a safe practice scenario for “Duck, Cover, and Hold.” A strong explanation should connect controls to the safety lesson rather than focusing only on graphics. The game demonstrates event-driven input, a real-time update loop, collision detection, scene transitions and score/result feedback.

5. Separate quiz module

The earthquake-quiz directory is an independent React/Vite project, not a Phaser scene. It contains scenario prompts and answer feedback. The main dashboard can link to earthquake learning pages and the drill, but build/dependency commands for the standalone quiz are separate.

6. Code review and testing prompts

- Check scene cleanup and keyboard event lifecycle when transitioning to results.
- Verify collision bounds match visible player and hazard assets.
- Check scaling on narrow screens and keyboard-only accessibility.
- Ensure the game communicates clear instructions and restart behavior.
- Test that no parent card/iframe creates unexpected nested scrolling.
- Do not claim gameplay progress persists to MongoDB unless an implemented persistence path is found.

7. Interview explanation

“The drill is a browser-side Phaser 3 game with separate scenes for setup, gameplay and results. Keyboard input changes player movement and cover behavior; the update loop advances hazards and collision checks determine safety outcomes. The standalone scenario quiz is a different React/Vite application.”